

LAPORAN PRAKTIKUM

▪ Identitas Praktikum

Nama MK : Praktikum Pemrograman Berorientasi Objek
Kode MK : CAK2KAB4
Bobot SKS : 3 SKS
Tempat : L-MM, Gedung IOT, lantai 3
Hari, tanggal : Selasa, 21 April 2026
Jam : 09:30-12:30 WIB
Topik praktikum : Modul-7

▪ Identitas Mahasiswa

Nama lengkap : Keishin Naufa Alfaridzhi
NIM : 103112400061
Program Studi : S-1 Teknik Informatika

▪ Algoritma & Penjelasan

Program ini merupakan implementasi dari konsep **Interface** dan **Abstract Class** dalam Pemrograman Berorientasi Objek, yang menggambarkan sistem pembayaran dengan berbagai metode. Terdapat satu interface *Pembayaran*, satu abstract class *MetodePembayaran*, serta tiga class konkret yaitu *DompotDigital*, *KartuKredit*, dan *RekeningBank*, ditambah satu class Main sebagai titik eksekusi program. Setiap metode pembayaran memiliki perilaku berbeda dalam memproses transaksi, namun semuanya mengikuti kontrak yang sama yang didefinisikan oleh interface Pembayaran.

▪ Kode Program & Penjelasan (Pembayaran.java)

```
package MODUL67.Abstract;  
  
import java.util.List;  
  
public interface Pembayaran {  
    void bayar(double jumlah);  
    void cekSaldo();  
    List<String> getRiwayat();  
}
```

Penjelasan:

Interface **Pembayaran** berfungsi sebagai kontrak yang wajib dipenuhi oleh semua class yang mengimplementasikannya. Untuk penjelasan rinci terkait interface ini adalah sebagai berikut:

1. Method bayar(double jumlah)

Mendefinisikan kontrak untuk melakukan transaksi pembayaran sejumlah nilai yang diberikan melalui parameter jumlah. Setiap class yang mengimplementasikan interface ini wajib menyediakan implementasi method ini sesuai logika pembayarannya masing-masing.

2. Method cekSaldo()

Mendefinisikan kontrak untuk menampilkan informasi saldo atau tagihan dari metode pembayaran yang bersangkutan. Implementasinya dapat berbeda-beda tergantung jenis metode pembayaran.

3. Method getRiwayat()

Mendefinisikan kontrak untuk mengembalikan daftar riwayat transaksi dalam bentuk List<String>. Method ini memungkinkan setiap metode pembayaran menyimpan dan menyediakan catatan seluruh transaksi yang telah dilakukan.

(MetodePembayaran.java)

```
package MODUL67.Abstract;  
  
import java.util.ArrayList;  
import java.util.List;  
  
public abstract class MetodePembayaran implements Pembayaran {  
    private String namaMetode;  
    private double saldo;  
    private List<String> riwayat;  
  
    public MetodePembayaran(String namaMetode, double saldoAwal) {  
        this.namaMetode = namaMetode;  
        this.saldo = saldoAwal;  
        this.riwayat = new ArrayList<>();  
    }  
  
    public abstract boolean validasiPembayaran(double jumlah);  
  
    public String getNamaMetode() {  
        return namaMetode;  
    }  
  
    public double getSaldo() {  
        return saldo;  
    }  
  
    public void setSaldo(double saldo) {  
        this.saldo = saldo;  
    }  
  
    public void tambahRiwayat(String catatan) {  
        riwayat.add(catatan);  
    }  
  
    @Override  
    public void cekSaldo() {
```

```

        System.out.println(namaMetode + " -> Rp " + (Long) saldo);
    }

    @Override
    public List<String> getRiwayat() {
        return riwayat;
    }
}

```

Penjelasan:

Abstract class **MetodePembayaran** merupakan class induk yang mengimplementasikan interface Pembayaran secara parsial, menyediakan atribut dan method umum yang digunakan bersama oleh seluruh metode pembayaran. Untuk penjelasan rinci terkait class ini adalah sebagai berikut:

1. Atribut

Terdapat atribut `namaMetode` bertipe `String` untuk menyimpan nama metode pembayaran, `saldo` bertipe `double` untuk menyimpan saldo atau total tagihan, serta `riwayat` bertipe `List<String>` untuk menyimpan catatan seluruh riwayat transaksi. Ketiganya bersifat `private`.

2. Constructor

Menerima dua parameter yaitu `namaMetode` dan `saldoAwal`, kemudian masing-masing ditetapkan ke atribut class menggunakan keyword `this`. Atribut `riwayat` diinisialisasi sebagai `ArrayList` kosong yang siap menampung catatan transaksi.

3. Method abstract `validasiPembayaran(double jumlah)`

Dideklarasikan sebagai abstract sehingga setiap subclass wajib mengimplementasikannya sesuai aturan validasinya masing-masing. Method ini digunakan untuk memeriksa apakah suatu transaksi dapat dilanjutkan atau tidak.

4. Getter dan Setter

Terdapat `getNamaMetode()` dan `getSaldo()` untuk mengakses nilai atribut `private` dari luar class, serta `setSaldo()` untuk memperbarui nilai saldo setelah transaksi berlangsung.

5. Method `tambahRiwayat(String catatan)`

Menambahkan satu entri catatan transaksi ke dalam list riwayat. Method ini dipanggil oleh subclass setiap kali sebuah transaksi berhasil maupun gagal diproses.

6. Method `cekSaldo()`

Di-*override* dari interface Pembayaran untuk menampilkan nama metode dan saldo saat ini ke konsol dalam format: `[namaMetode] -> Rp [saldo]`.

7. Method `getRiwayat()`

Di-*override* dari interface Pembayaran untuk mengembalikan list riwayat yang berisi seluruh catatan transaksi yang telah dicatat oleh subclass.

(DompetsDigital.java)

```
package MODUL67.Abstract;  
  
public class DompetsDigital extends MetodePembayaran {  
    public DompetsDigital(double saldoAwal) {  
        super("Dompets Digital", saldoAwal);  
    }  
  
    @Override  
    public boolean validasiPembayaran(double jumlah) {  
        return getSaldo() >= jumlah;  
    }  
  
    @Override  
    public void bayar(double jumlah) {  
        if (jumlah <= 0) {  
            tambahRiwayat("[GAGAL] " + getNamaMetode() + " - jumlah tidak valid: Rp " +  
(Long) jumlah);  
            return;  
        }  
  
        if (validasiPembayaran(jumlah)) {  
            setSaldo(getSaldo() - jumlah);  
            tambahRiwayat("[BERHASIL] " + getNamaMetode() + " bayar Rp " + (Long) jumlah  
                + " | sisa saldo: Rp " + (Long) getSaldo());  
        } else {  
            tambahRiwayat("[GAGAL] " + getNamaMetode() + " bayar Rp " + (Long) jumlah  
                + " | saldo saat ini: Rp " + (Long) getSaldo());  
        }  
    }  
}
```

Penjelasan:

Class **DompetsDigital** merupakan subclass dari MetodePembayaran yang merepresentasikan metode pembayaran menggunakan dompet digital berbasis saldo. Untuk penjelasan rinci terkait class ini adalah sebagai berikut:

1. Constructor

Memanggil constructor superclass menggunakan `super("Dompets Digital", saldoAwal)` untuk menetapkan nama metode dan saldo awal yang diberikan.

2. Method `validasiPembayaran(double jumlah)`

Di-*override* untuk memeriksa apakah saldo yang dimiliki saat ini mencukupi untuk membayar sejumlah jumlah. Mengembalikan `true` jika saldo lebih besar atau sama dengan jumlah yang akan dibayarkan.

3. Method `bayar(double jumlah)`

Di-*override* untuk mengimplementasikan logika pembayaran dompet digital. Pertama, memeriksa apakah jumlah bernilai nol atau negatif — jika iya, transaksi langsung dicatat sebagai gagal dengan keterangan jumlah tidak valid. Jika jumlah valid, method memanggil `validasiPembayaran()` untuk mengecek kecukupan saldo.

Apabila saldo mencukupi, saldo dikurangi sejumlah jumlah dan transaksi dicatat sebagai berhasil beserta sisa saldo. Apabila saldo tidak mencukupi, transaksi dicatat sebagai gagal beserta informasi saldo saat ini.

(KartuKredit.java)

```
package MODUL67.Abstract;

public class KartuKredit extends MetodePembayaran {
    private final double limitKredit;

    public KartuKredit(double limitKredit) {
        super("Kartu Kredit", 0);
        this.limitKredit = limitKredit;
    }

    @Override
    public boolean validasiPembayaran(double jumlah) {
        return (getSaldo() + jumlah) <= limitKredit;
    }

    @Override
    public void bayar(double jumlah) {
        if (jumlah <= 0) {
            tambahRiwayat("[GAGAL] " + getNamaMetode() + " - jumlah tidak valid: Rp " +
(Long) jumlah);
            return;
        }

        if (validasiPembayaran(jumlah)) {
            setSaldo(getSaldo() + jumlah);
            tambahRiwayat("[BERHASIL] " + getNamaMetode() + " transaksi Rp " + (Long)
jumlah
                + " | total tagihan: Rp " + (Long) getSaldo()
                + " | limit: Rp " + (Long) limitKredit);
        } else {
            tambahRiwayat("[GAGAL] " + getNamaMetode() + " transaksi Rp " + (Long)
jumlah
                + " | tagihan saat ini: Rp " + (Long) getSaldo()
                + " | limit: Rp " + (Long) limitKredit);
        }
    }

    @Override
    public void cekSaldo() {
        System.out.println(getNamaMetode() + " -> tagihan: Rp " + (Long) getSaldo()
            + " / limit: Rp " + (Long) limitKredit);
    }
}
```

Penjelasan:

Class **KartuKredit** merupakan subclass dari MetodePembayaran yang merepresentasikan metode pembayaran menggunakan kartu kredit dengan sistem akumulasi tagihan. Untuk penjelasan rinci terkait class ini adalah sebagai berikut:

1. Atribut

Terdapat atribut tambahan `limitKredit` bertipe `double` yang bersifat final, digunakan untuk menyimpan batas maksimal total tagihan yang diizinkan. Atribut ini tidak dapat diubah setelah diinisialisasi.

2. Constructor

Memanggil constructor superclass dengan nama "Kartu Kredit" dan saldo awal 0, karena pada kartu kredit saldo digunakan untuk melacak total tagihan yang bertambah setiap ada transaksi. Atribut `limitKredit` ditetapkan dari parameter yang diberikan.

3. Method `validasiPembayaran(double jumlah)`

Di-*override* untuk memeriksa apakah total tagihan saat ini ditambah jumlah transaksi baru tidak melebihi `limitKredit`. Mengembalikan `true` jika total tersebut masih dalam batas limit yang diizinkan.

4. Method `bayar(double jumlah)`

Di-*override* untuk mengimplementasikan logika transaksi kartu kredit. Berbeda dengan dompet digital, saldo pada kartu kredit justru *bertambah* setiap transaksi berhasil karena merepresentasikan akumulasi tagihan. Apabila transaksi valid dan tidak melewati limit, saldo ditambah sejumlah jumlah dan dicatat sebagai berhasil beserta total tagihan dan limit. Apabila melebihi limit, transaksi dicatat sebagai gagal.

5. Method `cekSaldo()`

Di-*override* kembali dari implementasi superclass untuk menampilkan informasi yang lebih spesifik berupa total tagihan saat ini dan limit kartu dalam format: `[namaMetode] -> tagihan: Rp [tagihan] / limit: Rp [limit]`.

(RekeningBank.java)

```
package MODUL67.Abstract;

public class RekeningBank extends MetodePembayaran {
    private static final double BIAYA_ADMIN = 2500;

    public RekeningBank(double saldoAwal) {
        super("Rekening Bank", saldoAwal);
    }

    @Override
    public boolean validasiPembayaran(double jumlah) {
        return getSaldo() >= (jumlah + BIAYA_ADMIN);
    }

    @Override
    public void bayar(double jumlah) {
        if (jumlah <= 0) {
            tambahRiwayat("[GAGAL] " + getNamaMetode() + " - jumlah tidak valid: Rp " +
(Long) jumlah);
        }
    }
}
```

```

        return;
    }

    double totalPotong = jumlah + BIAYA_ADMIN;

    if (validasiPembayaran(jumlah)) {
        setSaldo(getSaldo() - totalPotong);
        tambahRiwayat("[BERHASIL] " + getNamaMetode() + " bayar Rp " + (Long) jumlah
            + " | biaya admin: Rp " + (Long) BIAYA_ADMIN
            + " | sisa saldo: Rp " + (Long) getSaldo());
    } else {
        tambahRiwayat("[GAGAL] " + getNamaMetode() + " bayar Rp " + (Long) jumlah
            + " | butuh: Rp " + (Long) totalPotong
            + " | saldo saat ini: Rp " + (Long) getSaldo());
    }
}
}

```

Penjelasan:

Class **RekeningBank** merupakan subclass dari MetodePembayaran yang merepresentasikan metode pembayaran menggunakan rekening bank dengan pengenaan biaya administrasi pada setiap transaksi. Untuk penjelasan rinci terkait class ini adalah sebagai berikut:

1. Atribut

Terdapat atribut tambahan BIAYA_ADMIN bertipe double yang bersifat static final dengan nilai tetap 2500. Karena bersifat *static*, atribut ini dimiliki oleh class itu sendiri dan berlaku sama untuk seluruh objek RekeningBank.

2. Constructor

Memanggil constructor superclass dengan nama "Rekening Bank" dan saldoAwal yang diberikan untuk menetapkan saldo awal rekening.

3. Method validasiPembayaran(double jumlah)

Di-*override* untuk memeriksa apakah saldo yang tersedia mencukupi untuk membayar jumlah transaksi ditambah BIAYA_ADMIN. Artinya, saldo minimal yang dibutuhkan adalah jumlah transaksi plus Rp 2.500.

4. Method bayar(double jumlah)

Di-*override* untuk mengimplementasikan logika pembayaran rekening bank. Terlebih dahulu menghitung totalPotong yaitu jumlah transaksi ditambah biaya admin. Apabila saldo mencukupi, saldo dikurangi sejumlah totalPotong dan transaksi dicatat sebagai berhasil beserta rincian biaya admin dan sisa saldo. Apabila tidak mencukupi, transaksi dicatat sebagai gagal beserta informasi total yang dibutuhkan dan saldo saat ini.

(Main.java)

```
package MODUL67.Abstract;  
  
import java.util.List;  
  
public class Main {  
    public static void main(String[] args) {  
        DompetsDigital dompet = new DompetsDigital(150000);  
        KartuKredit kartu = new KartuKredit(5000000);  
        RekeningBank rekening = new RekeningBank(300000);  
  
        dompet.bayar(50000);  
        dompet.bayar(120000);  
  
        kartu.bayar(2000000);  
        kartu.bayar(3500000);  
  
        rekening.bayar(100000);  
        rekening.bayar(200000);  
  
        System.out.println("Cek Saldo & Tagihan:");  
        dompet.cekSaldo();  
        kartu.cekSaldo();  
        rekening.cekSaldo();  
  
        tampilkanRiwayat("Dompets Digital", dompet.getRiwayat());  
        tampilkanRiwayat("Kartu Kredit", kartu.getRiwayat());  
        tampilkanRiwayat("Rekening Bank", rekening.getRiwayat());  
    }  
  
    private static void tampilkanRiwayat(String judul, List<String> riwayat) {  
        System.out.println("\nRiwayat " + judul + ":");  
        for (String item : riwayat) {  
            System.out.println(item);  
        }  
    }  
}
```

Penjelasan:

Class **Main** merupakan titik eksekusi program yang mendemonstrasikan penggunaan seluruh metode pembayaran yang telah dibuat. Untuk penjelasan rinci terkait class ini adalah sebagai berikut:

1. Inisialisasi Objek

Tiga objek dibuat: dompet bertipe DompetsDigital dengan saldo awal Rp 150.000, kartu bertipe KartuKredit dengan limit Rp 5.000.000, dan rekening bertipe RekeningBank dengan saldo awal Rp 300.000.

2. Simulasi Transaksi

Setiap metode pembayaran diuji dengan dua skenario transaksi. Dompets digital mencoba membayar Rp 50.000 (berhasil) dan Rp 120.000 (gagal karena saldo tidak cukup). Kartu kredit mencoba transaksi Rp 2.000.000 (berhasil) dan Rp 3.500.000 (gagal karena melebihi limit). Rekening bank mencoba membayar

Rp 100.000 (berhasil) dan Rp 200.000 (gagal karena saldo tidak mencukupi termasuk biaya admin).

3. Menampilkan Saldo & Tagihan

Method `cekSaldo()` dipanggil pada masing-masing objek untuk menampilkan kondisi saldo atau tagihan terkini setelah seluruh simulasi transaksi dijalankan.

4. Method `tampilkanRiwayat()`

Merupakan method *helper* private yang menerima judul dan list riwayat, kemudian mencetak seluruh entri riwayat transaksi ke konsol. Method ini dipanggil tiga kali untuk menampilkan riwayat lengkap dari masing-masing metode pembayaran.

▪ Hasil Running Program

```
Cek Saldo & Tagihan:
Dompot Digital -> Rp 100000
Kartu Kredit -> tagihan: Rp 2000000 / limit: Rp 5000000
Rekening Bank -> Rp 197500

Riwayat Dompot Digital:
[BERHASIL] Dompot Digital bayar Rp 50000 | sisa saldo: Rp 100000
[GAGAL] Dompot Digital bayar Rp 120000 | saldo saat ini: Rp 100000

Riwayat Kartu Kredit:
[BERHASIL] Kartu Kredit transaksi Rp 2000000 | total tagihan: Rp 2000000 | limit: Rp 5000000
[GAGAL] Kartu Kredit transaksi Rp 3500000 | tagihan saat ini: Rp 2000000 | limit: Rp 5000000

Riwayat Rekening Bank:
[BERHASIL] Rekening Bank bayar Rp 100000 | biaya admin: Rp 2500 | sisa saldo: Rp 197500
[GAGAL] Rekening Bank bayar Rp 200000 | butuh: Rp 202500 | saldo saat ini: Rp 197500
```

▪ Link Program Praktikum (Github)

[kukingkux/PRAKTIKUM-PBO](https://github.com/kukingkux/PRAKTIKUM-PBO)